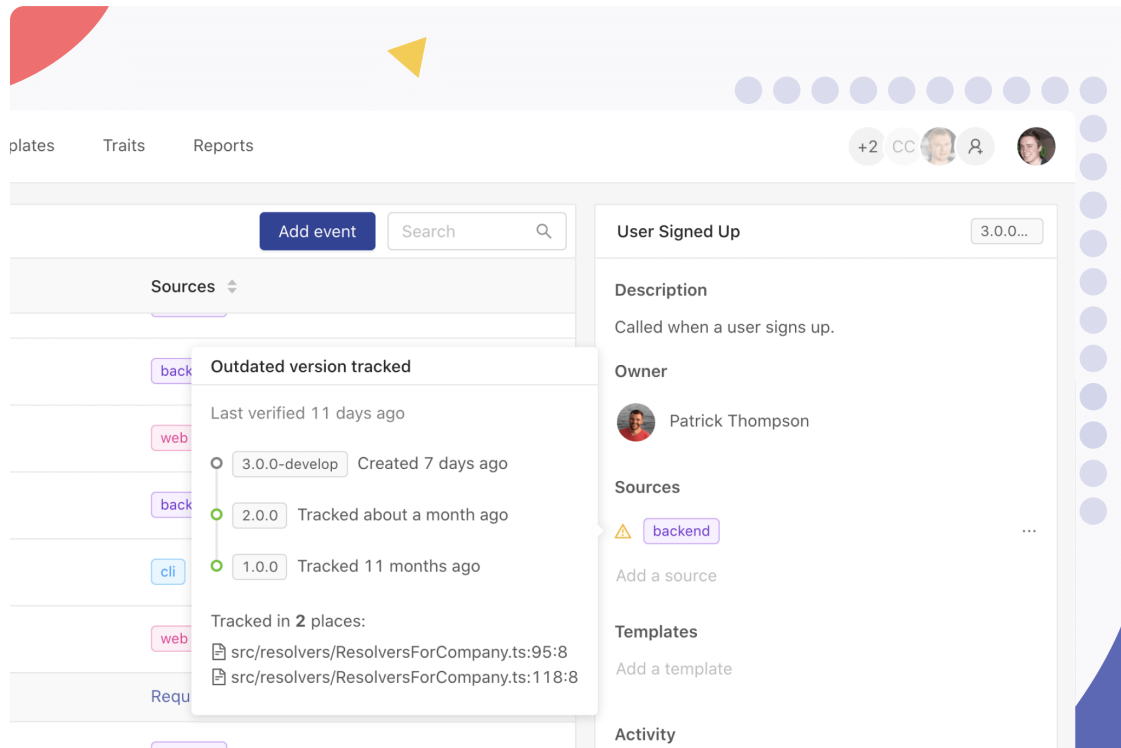


Introducing versioning

To treat analytics like code, one must treat analytics schema like code. How does that work in Iteratively?



Ondrej Hrebicek

June 22, 2020

7 min read

Contents

[Wrangling your analytics](#)[Tracking plan versioning](#)[Event versioning](#)[Conclusion](#)

To support a growing number of enterprise customers relying on Iteratively to govern their analytics schema, we're revamping our platform and implementing robust, Git-like versioning directly in the product.

Wrangling your analytics

1. More people get involved and contribute changes to the tracking plan
2. The tracking plan grows and begins to evolve in unexpected ways

At first, tracking plans feel deceptively simple. A Google Sheet, Airtable, or even a Confluence page gets the job done — one person defines the app's key events, another person instruments them in the product.

Others on the team begin to notice the stream of valuable data coming in and ask for more. What would it take to instrument a new feature or experiment? Can we also capture X? Please remove event Y, we no longer need it.

Soon, what used to be a simple document begins to take on complexity. A new column to track the person responsible. Another to list expected values. Developers start to mark events implemented or WIP, and add another column to note the app version the event shipped in. Someone overwrites someone else's changes so next time that person makes a copy of the spreadsheet first. And throughout all this, events and properties are being added, renamed, and deleted.

To help companies tackle this chaotic nature of tracking plan change management, we are adding support for:

1. Tracking plan versioning, with support for staging areas and parallel branches so teams can work side-by-side and merge their work when ready
2. Granular versioning of individual events so changes to their schema (shape) are explicit and clearly visible to everyone on the team

Tracking plan versioning

We are building Iteratively to be like a Git repo for a team's analytics schema. Tracking plans are implemented in code and interpreted by code, so behind the scenes, we are treating them as code.

Going forward, every company account is provisioned with its own tracking plan version tree. A tree represents the evolution of a company's tracking plan, and each node in the tree represents a tracking plan version. We track exactly how each version came about — what the plan looked like before and after — and can visualize this progression to help teams understand the origins of their analytics data.

Tracking plan versions get created when a team is happy with a proposed set of tracking plan changes and decides to publish them. Up to that point, the team works on those changes in a separate staging area without disrupting anyone else. By default, developers don't see those changes, so they can't accidentally implement them in the

Companies with multiple teams working on multiple features in parallel typically create source code branches to isolate their efforts until they're ready to combine their work. These teams can now match this workflow in Iteratively by creating new branches for their tracking plan as well. Just like in source code, a branch is a parallel tracking plan to be iterated on in isolation and eventually merged back into the main branch. This allows developers to instrument analytics code that matches their team's proposed tracking plan without affecting anyone else.

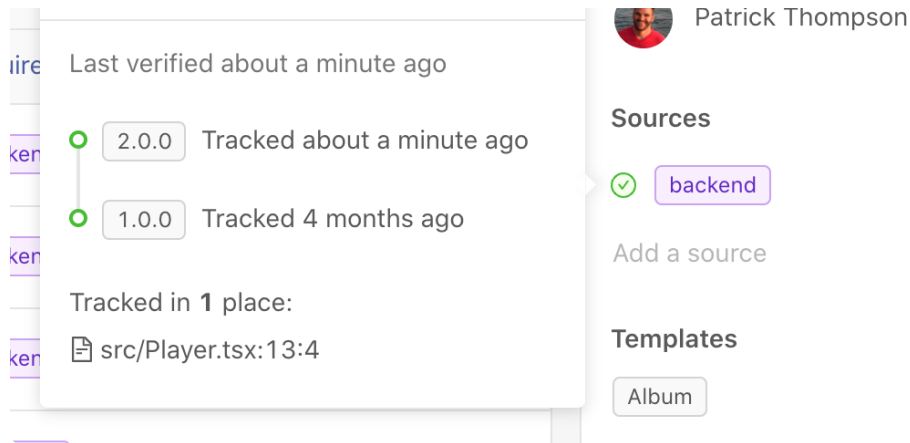
Similar to Git, Iteratively's branches support:

- Syncing of branches using pull and merge, to keep branches up-to-date and facilitate bi-directional workflow
- Merge requests, to initiate a review (and optionally approval by a set number of reviewers) of changes prior to a merge
- Conflict detection & resolution, to handle cases where multiple users may have made different changes to the same events or properties
- Ability to visualize the exact differences between two branches and to highlight which changes are backwards compatible

When itly, the Iteratively CLI reports on a tracking plan's implementation status in source code, that information is now associated with the corresponding tracking plan version as well. For example, if a developer instruments tracking plan version 2 but tracking plan version 3 has since been published, the reported implementation status will still affect version 2 only. This means that at any time, anyone at the company can understand:

- Which tracking plan versions are implemented on which platforms (web, iOS, etc.) and whether all shipped platforms are in sync
- When have devs implemented a version on a platform — in other words, when did the team begin collecting data defined in a particular tracking plan version
- How a platform's tracking evolved over time — which versions were implemented when, and whether at any time (perhaps due to a code rollback) a platform began tracking an older version again

Event versioning



If we think of —

- A tracking plan as a Git repository
- A tracking plan version as a Git commit
- A tracking plan version's contents as a Git directory

Then an analytics event is like a Git file, stored de-duplicated with its own history and metadata.

Just like in Git, versioning individual events serves a different purpose from versioning tracking plans.

A tracking plan version:

- Uniquely identifies a point-in-time collection of event versions
- Matches one-for-one a particular version of a codegen'd SDK
- Can be tied to a particular product release

An event version:

- Uniquely identifies a particular event schema
- Is referenced when an event is found or not found in source code
- Can be linked to downstream processing & storage, for example a DB table

A new tracking plan version is created every time a tracking plan is published with changed events, templates or properties. On the other hand, a new event version is created every time an event changes because the event itself, its properties, its templates or its templates' properties change.

Event changes are detected when a new tracking plan version is published. If the event is found to have changed in a new tracking plan version, a new event version is created and a new version number is assigned.

interact with previously collected data — in other words, whether the schema would validate such data.

In practice, this means that Iteratively assigns a version number in the form model-revision-addition[-branch] to every event schema version, where:

- model is incremented for breaking changes incompatible with previously collected data
- revision is always 0 — there is no use case today that would cause a revision increment
- addition is incremented for non-breaking changes compatible with previously collected data
- branch is temporarily appended for schemas tracked in development branches of the tracking plan

When creating a new event schema version, Iteratively automatically detects its backwards compatibility and assigns a new version number accordingly. When a user is preparing to publish a new tracking plan version, it also highlights changes that are backwards incompatible to alert them that a particular change may break their existing analytics configuration.

Finally, because Iteratively tracks event versions separately from tracking plan versions, it is able to accurately report on an event's implementation status even when a new tracking plan version is created. For example, an event that hasn't changed in a new tracking plan version continues to report as implemented, even though the instrumented SDK matches an older version of the plan. An event's version, not just the tracking plan version, is therefore taken into account.

Conclusion

We hope this post shed more light on how versioning works in Iteratively and explained the role tracking plan and event versioning plays in any analytics team's arsenal. As always, we'd love to hear your feedback and any ideas that would improve how you use Iteratively. You can send us feedback in-product or by getting in touch with us at team@iterative.ly.

SIGN UP FOR PRODUCT UPDATES & RELATED CONTENT

TRUSTED BY GREAT TEAMS LIKE

ARTIFACT
UPRISING

box

THREDUP

dribbble

octopus



Product

Customers

Pricing

Docs

Blog

Log in

Sign up

Pricing

Blog

GitHub

Careers

Integrations

Contact us

Security

Legal